

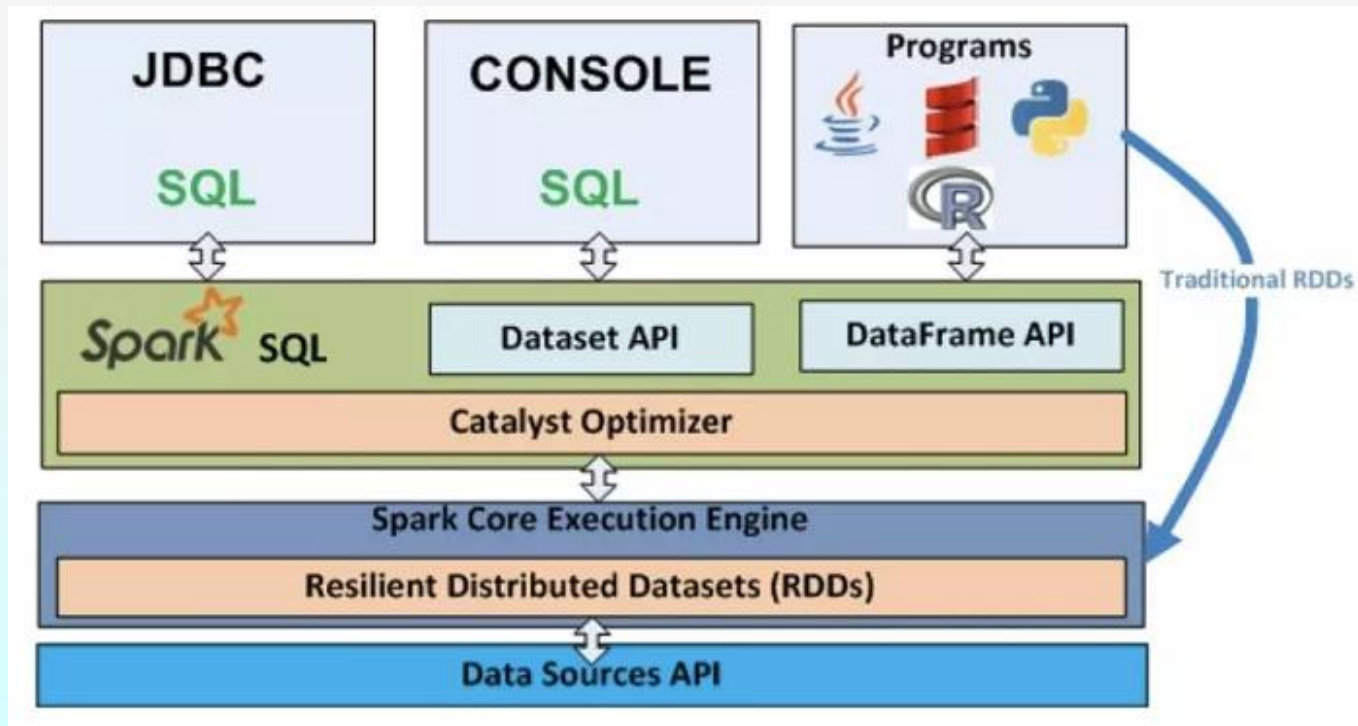
Hadoop与Spark

SparkSQL



SparkSQL

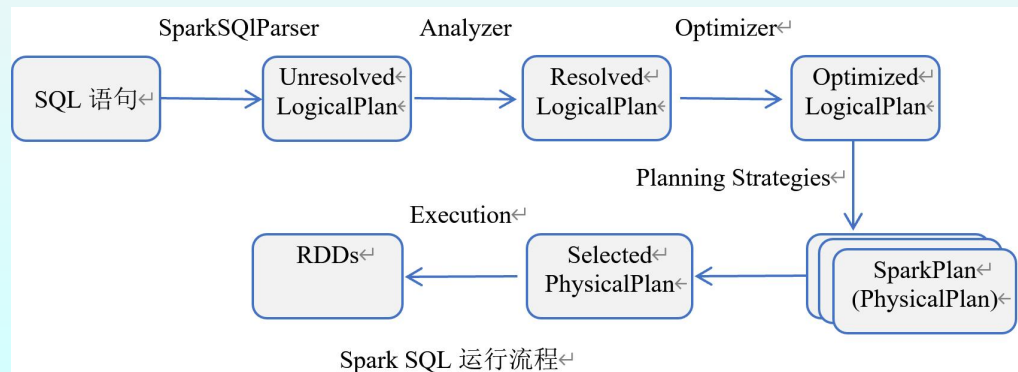
- ◆ SparkSQL简介
 - ◆ Spark SQL是Spark平台最重要的组件之一
 - ◆ 它提供最新的DataFrame API，支持SQL查询
 - ◆ Spark SQL的核心是Catalyst优化器，Catalyst支持基于规则和基于成本的查询优化



SparkSQL

◆ SparkSQL的SQL查询执行流程

- ◆ (1) **Parser**将SQL语句通过词法和语法解析，生成未解析的逻辑执行计划(Unresolved Logical Plan)，然后在后续步骤中使用不同的Rule，应用到该逻辑计划上
- ◆ (2) **Analyzer**使用各种分析规则，配合元数据，对未解析的逻辑计划进行处理，转换成解析的逻辑计划
- ◆ (3) **Optimizer**使用各种优化规则，对解析的逻辑计划，进行合并、列裁剪、和过滤器下推等优化工作，生成优化的逻辑计划
- ◆ (4) **Planner**使用Planning Strategies，对优化的逻辑计划进行转换，根据统计信息和代价模型，生成可以执行的物理执行计划，选择最优的物理执行计划，得到Spark Plan
- ◆ (5) 最后，真正执行物理执行计划，调用Spark Plan的**execute()**方法，对RDD进行处理，获得查询结果



pause



SparkSQL

- ◆ SparkSQL简介
 - ◆ 1, 启动pyspark, 使用Python语言查询本地文件JSON
 - ◆ JSON文件包含了元信息, 其内容装载到一个DataFrame。这个DataFrame注册为一个临时视图, 然后在上面运行SQL

```
from pyspark.sql import SparkSession

spark = SparkSession \
    .builder \
    .appName("Python Spark SQL basic example") \
    .config("spark.some.config.option", "some-value") \
    .getOrCreate()

df =
spark.read.json("file:///opt/linuxsir/spark/examples/src/main/resources/people.json")
df.show()

df.createOrReplaceTempView("people")
sqlDF = spark.sql("SELECT * FROM people")
sqlDF.show()
```

SparkSQL

◆ SparkSQL简介

◆ 2, 启动pyspark, 使用Python语言查询本地文件TXT

◆ 从本地TXT文件读取内容, 并且进行查询处理。people.txt包含两列, 分别是name列和age列, 在代码中需要为文本文件动态建立模式Schema, 用这个模式解释每行数据, 建立DataFrame。这个DataFrame注册为一个临时视图, 然后在上面运行SQL

```
from pyspark.sql.types import *
sc = spark.sparkContext

lines = sc.textFile("file:///opt/linuxsir/spark/examples/src/main/resources/people.txt")
parts = lines.map(lambda l: l.split(","))
people = parts.map(lambda p: (p[0], p[1].strip()))

schemaString = "name age"
fields = [StructField(field_name, StringType(), True) for field_name in schemaString.split()]
schema = StructType(fields)
schemaPeople = spark.createDataFrame(people, schema)

schemaPeople.createOrReplaceTempView("people")
results = spark.sql("SELECT name FROM people")
results.show()
```

SparkSQL

- ◆ SparkSQL简介
 - ◆ 3, 启动pyspark, 使用Python语言查询HDFS Parquet格式文件
 - ◆ 从JSON读取数据, 写入HDFS的Parquet文件[]。然后再把Parquet文件读进来, 进行查询处理

```
from pyspark.sql.types import *
sc = spark.sparkContext

peopleDF =
spark.read.json("file:///opt/linuxsir/spark/examples/src/main/resources/people.json")
peopleDF.show()
peopleDF.write.parquet("/input/people.parquet")

parquetFile = spark.read.parquet("/input/people.parquet")
parquetFile.createOrReplaceTempView("parquetFile")
teenagers = spark.sql("SELECT name FROM parquetFile WHERE age >= 13 AND age <= 19")
teenagers.show()

quit()
```

SparkSQL

- ◆ SparkSQL简介
 - ◆ 3, 启动pyspark, 使用Python语言查询HDFS Parquet格式文件
 - ◆ 退出pyspark, 可以检查HDFS上的parquet文件/input/people.parquet是否生成了。注意, /input/people.parquet是一个目录

```
cd /opt/linuxsir/hadoop/bin  
./hdfs dfs -ls /input
```

可以把这个文件删除, 然后再实验上述代码。

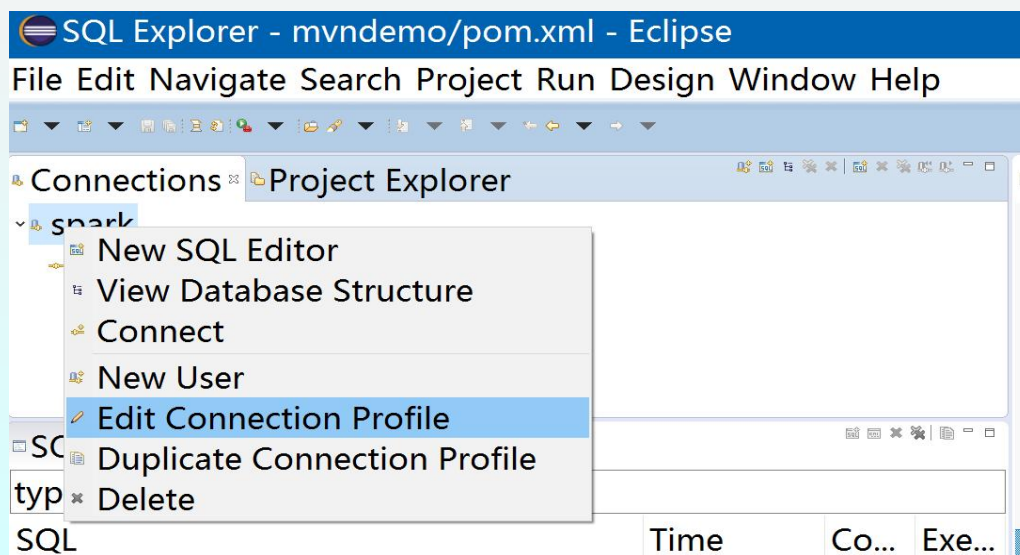
```
cd /opt/linuxsir/hadoop/bin  
./hdfs dfs -rmr /input/people.parquet
```

pause



SparkSQL

- ◆ SparkSQL简介
 - ◆ 通过SQL Explorer插件存取SparkSQL
 - ◆ 我们可以在Windows上，为Eclipse安装SQL Explorer插件，然后通过SQL Explorer连接Spark Thrift Server，执行SQL查询。需要为Eclipse的SQL Explorer插件，配置Driver Class、URL、Jars包等。
 - ◆ 打开SQL Explorer(Eclipse的一个plugin，需要事先安装这个plugin)的Connections视图，新建连接，并对连接进行属性设置；如果已经建立连接，可以对连接按右键，选择“Edit Connection Profile”，如图所示



SparkSQL

- ◆ SparkSQL简介
 - ◆ 通过SQL Explorer插件存取SparkSQL
 - ◆ 连接的属性设置对话框，如图所示
 - ◆ 需要设置连接的name(名称)、Driver、URL、用户名、密码等属性

Change Connection Profile

Change Connection Profile

Name: spark

Driver: spark_driver Add/Edit Drivers

URL: jdbc:hive2://hd-master:10013

☐ User name is not required for this database

☒ Auto Login during startup ☐ Logon during startup

User: root

Password: *****

☐ AutoCommit ☐ Commit On Close

OK Cancel

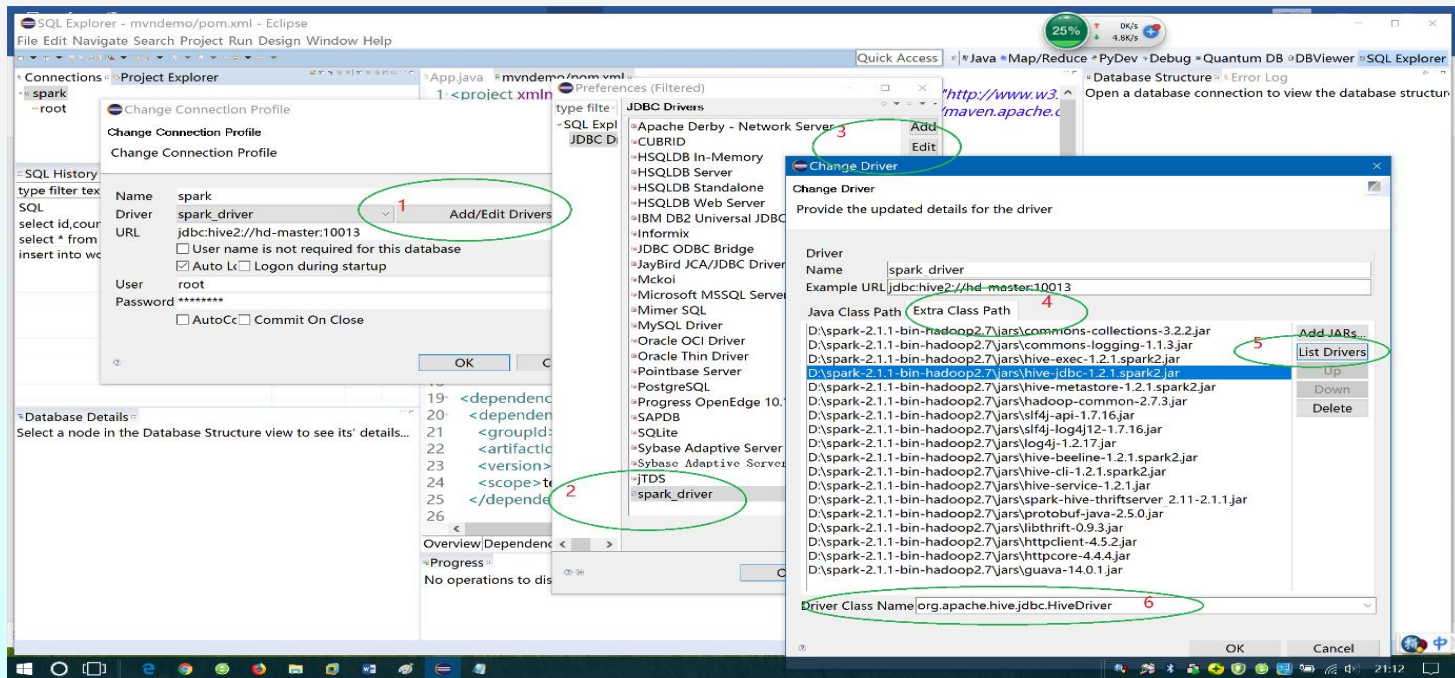
SparkSQL

◆ SparkSQL简介

◆ 通过SQL Explorer插件存取SparkSQL

◆ 接着对Spark Driver进行设置，可以参考下图按照序号给出的操作步骤，进行设置。这些操作的目的是新建或者编辑一个Driver类型，并且导入相关的Jar包，指定driver class name等

◆ 我们把Spark安装包解压缩在Windows的d:盘上，把d:\spark-2.1.1-bin-hadoop2.7\jar目录下的jar包导入即可



pause



SparkSQL

- ◆ SparkSQL简介
 - ◆ JDBC Java编程
 - ◆ 对于Java编程，和其它数据库的JDBC编程差别不大。只不过需要为项目配置Spark的Jar包。另外在程序中需要对Driver class、URL、user name和password进行设置
 - ◆ 完整的Java代码如下
 - ◆ 这个实例首先建立到Spark Thrift Server的连接，然后执行SQL，显示查询结果

```
package com.pai.mvndemo;  
  
import java.sql.Connection;  
import java.sql.DriverManager;  
import java.sql.ResultSet;  
import java.sql.SQLException;  
import java.sql.Statement;
```

SparkSQL

- ◆ SparkSQL简介

- ◆ JDBC Java编程

- ◆ 完整的Java代码如下。这个实例首先建立到Spark ThriftServer的连接，然后执行SQL，显示查询结果

```
public class App {  
    public static void main(String[] args) throws SQLException {  
        Connection conn;  
        Statement stmt;  
        ResultSet res;  
        String sql;  
        String url = "jdbc:hive2://192.168.31.129:10013/default";  
        try {  
            Class.forName("org.apache.hive.jdbc.HiveDriver");  
        } catch (ClassNotFoundException e) {  
            // TODO Auto-generated catch block  
            e.printStackTrace();  
        }  
        conn = DriverManager.getConnection(url, "root", "rootroot");  
        stmt = conn.createStatement();  
  
        sql = "SELECT * FROM words";  
  
        System.out.println("Running" + sql);  
        res = stmt.executeQuery(sql);  
        while (res.next()) {  
            System.out.println("id: " + res.getInt(1) + "\tword: " + res.getString(2));  
        }  
    }  
}
```

SparkSQL

- ◆ SparkSQL简介
 - ◆ JDBC Java编程
 - ◆ 如果用Maven进行Java编程，利用pom.xml文件，很方便进行项目依赖的设置，无需用户为项目导入jar包

```
<project
  xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-
4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.pai</groupId>
  <artifactId>mvndemo</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <packaging>jar</packaging>

  <name>mvndemo</name>
  <url>http://maven.apache.org</url>

  <properties>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>
```

SparkSQL

- ◆ SparkSQL简介
 - ◆ JDBC Java编程
 - ◆ 如果用Maven进行Java编程，利用pom.xml文件，很方便进行项目依赖的设置，无需用户为项目导入jar包

```
<dependencies>
<dependency>
  <groupId>junit</groupId>
  <artifactId>junit</artifactId>
  <version>3.8.1</version>
  <scope>test</scope>
</dependency>

<dependency>
  <groupId>org.apache.hive</groupId>
  <artifactId>hive-jdbc</artifactId>
  <version>1.2.1</version>
</dependency>

<dependency>
  <groupId>org.apache.hadoop</groupId>
  <artifactId>hadoop-common</artifactId>
  <version>2.7.3</version>
</dependency>

<dependency>
  <groupId>jdk.tools</groupId>
  <artifactId>jdk.tools</artifactId>
  <version>1.8</version>
  <scope>system</scope>
  <systemPath>C:/Program Files/Java/jdk1.8.0_131/lib/tools.jar</systemPath>
</dependency>

</dependencies>

</project>
```

SparkSQL

- ◆ SparkSQL简介
 - ◆ JDBC Java编程
 - ◆ 关于如何安装Maven，给Eclipse安装和配置Maven插件，以及新建Maven项目，请参考教材8.4节
 - ◆ 为了运行和调试该代码，应该在集群上启动HDFS、YARN、Hive metastore、Spark、以及SparkSQL Thrift Server
 - ◆ 结束以后，需要按照逆序，停止SparkSQL Thrift Server、Spark、Hive metastore、YARN、HDFS

pause



SparkSQL

- ◆ SparkSQL简介
 - ◆ 启动命令
 - ◆ 启动HDFS和YARN

```
rm -rf /opt/linuxsir/hadoop/logs/*.*  
ssh root@192.168.31.130 rm -rf /opt/linuxsir/hadoop/logs/*.*  
ssh root@192.168.31.131 rm -rf /opt/linuxsir/hadoop/logs/*.*
```

\\分开启动

```
clear
```

```
cd /opt/linuxsir/hadoop/sbin
```

```
./start-dfs.sh
```

```
./start-yarn.sh
```

SparkSQL

- ◆ SparkSQL简介
 - ◆ 启动命令
 - ◆ 检查各个节点上相关进程是否正确启动

```
clear  
jps  
ssh root@192.168.31.130 jps  
ssh root@192.168.31.131 jps
```

SparkSQL

- ◆ SparkSQL简介
 - ◆ 启动命令
 - ◆ 启动hive metastore, 并且检查进程已经启动和绑定到特定端口

```
cd /opt/linuxsir/hive  
./bin/hive --service metastore -p 19083 &
```

```
\\ 等待metastore启动  
netstat -lanp | grep 19083
```

SparkSQL

- ◆ SparkSQL简介
 - ◆ 启动命令
 - ◆ 启动spark, 并且检查各个节点上相关进程是否正确启动

```
cd /opt/linuxsir/spark  
./sbin/start-all.sh
```

```
jps  
ssh root@192.168.31.130 jps  
ssh root@192.168.31.131 jps
```

SparkSQL

- ◆ SparkSQL简介
 - ◆ 启动命令
 - ◆ 启动pyspark

```
cd /opt/linuxsir/spark  
MASTER=spark://hd-master:17077 ./bin/pyspark
```

SparkSQL

- ◆ SparkSQL简介
 - ◆ 停止命令
 - ◆ 停止spark

```
cd /opt/linuxsir/spark  
./sbin/stop-all.sh
```

SparkSQL

- ◆ SparkSQL简介
 - ◆ 停止命令
 - ◆ 停止metastore

```
netstat -lanp | grep 19083  
kill -9 16336 \\16336是metastore进程号
```

SparkSQL

- ◆ SparkSQL简介
 - ◆ 停止命令
 - ◆ 停止YARN和HDFS

```
cd /opt/linuxsir/hadoop/sbin  
./stop-yarn.sh  
./stop-dfs.sh
```

pause

